

Task 1: Image rotation with CUDA

Three-minute presentation and study companion

This guide contains the scripts for the six main slides, answers to all seven workshop reflection questions, the proposed image-rotation design, references, and the AI declaration and prompt record.

The design keeps the original image dimensions and fills uncovered pixels with black, as selected by the student. The accompanying PowerPoint contains editable diagrams and five clearly labelled appendix slides.

Presentation timing

Main slide	Seconds
1. Image rotation with CUDA	30
2. One thread computes one output pixel	35
3. Threads, blocks and the image grid	30
4. Millions of threads share finite hardware	25
5. Image size and transfer overhead	30
6. GPUDirect Storage and image rotation	30
Total	180

The scripts contain approximately 385 words. Timing is a rehearsal target, not a measured speaking duration. Practise aloud and leave room to explain the diagrams.

The latest confirmed timing is three minutes each: six minutes total, within the assessment maximum of seven minutes. The scripts leave space for pointing to diagrams and natural pauses.

Slide 1: Image rotation with CUDA

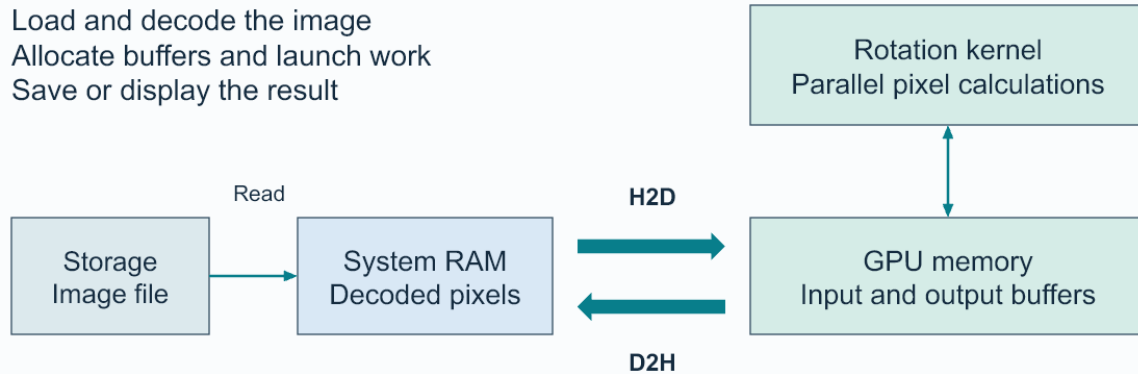
Suggested speaking time: 30 seconds

Image rotation with CUDA

Host: CPU

Load and decode the image
Allocate buffers and launch work
Save or display the result

Device: GPU



PCIe carries copies between RAM and GPU memory

Sources: [1] [2] [3]. Full references in appendix and companion PDF.

1

Speaker script

I'll explain how we'd rotate a large image using CUDA. The CPU, or host, loads and decodes the image, allocates buffers and launches the work. The GPU, or device, calculates the output pixels. In this design, system RAM and GPU memory are separate. We copy the pixels across PCIe using a host-to-device `cudaMemcpy`, then copy the result back. Those transfers take time, even when the rotation kernel is fast.

Sources: [1], [2], [3].

Slide 2: One thread computes one output pixel

Suggested speaking time: 35 seconds

One thread computes one output pixel

Brightness

Same independent operation on every pixel

`out = clamp(in + value, 0, 255)`

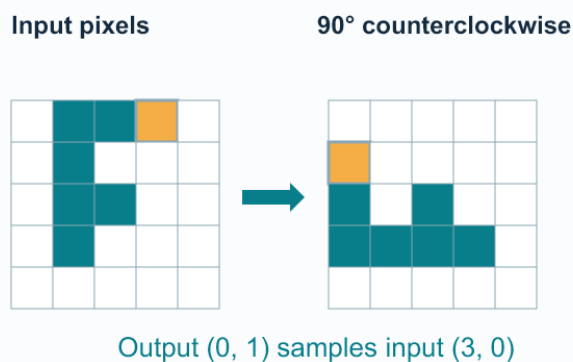
Rotation

Each output pixel independently finds and samples its source

Inverse mapping about the centre. Nearest neighbour. Fixed canvas. Black outside.

Sources: [1] [4] [5]. Full references in appendix and companion PDF.

2



Speaker script

Brightness is data-parallel because each pixel gets the same independent adjustment. Rotation also gives each thread independent work, but the source position changes. We start with an output pixel and map backwards to its source, so every destination gets written. Here, that produces a ninety-degree counterclockwise rotation. We use nearest-neighbour sampling, keep the original dimensions and fill uncovered pixels with black. It's simple, but diagonal edges can look jagged and corners can be cropped.

Sources: [1], [4], [5].

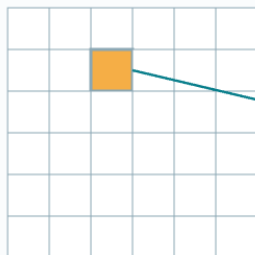
Slide 3: Threads, blocks and the image grid

Suggested speaking time: 30 seconds

Threads, blocks and the image grid

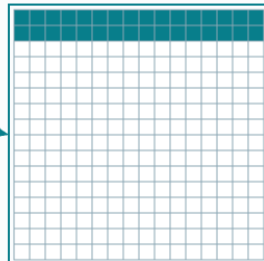
64 × 64 blocks

Grid view is schematic



16 × 16 threads

One block = 256 threads



Highlighted: one 32-thread warp

4,096 blocks

× 256 threads

= 1,048,576

launched threads

```
rotate<<<dim3(64,64), dim3(16,16)>>>(...);
```

Sources: [1] [2]. Full references in appendix and companion PDF.

3

Speaker script

CUDA organises threads into blocks, and blocks into a grid. The launch here specifies both. For a one-thousand-and-twenty-four square image, sixteen-by-sixteen threads gives two hundred and fifty-six threads per block. We need sixty-four-by-sixty-four blocks: four thousand and ninety-six blocks, launching just over one million threads. Our bounds check handles images that don't divide evenly by sixteen.

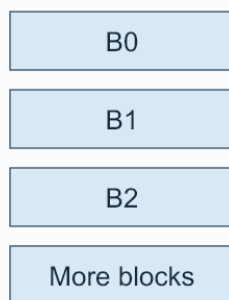
Sources: [1], [2].

Slide 4: Millions of threads share finite hardware

Suggested speaking time: 25 seconds

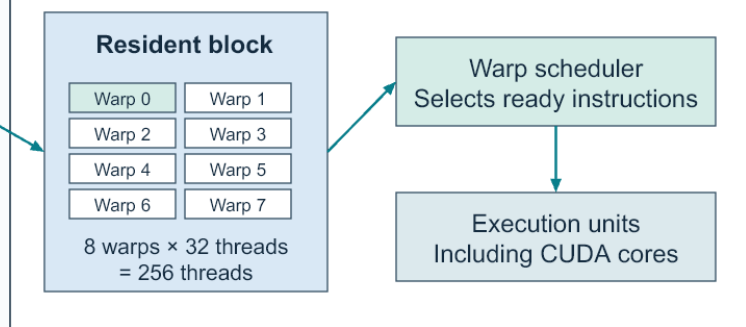
Millions of threads share finite hardware

Pending blocks



Blocks enter as resources become available

One streaming multiprocessor (SM)



Many SMs execute work concurrently. Core reuse continues over time.

Sources: [2] [6]. Full references in appendix and companion PDF.

4

Speaker script

That doesn't mean we need a million CUDA cores. An SM, or streaming multiprocessor, keeps blocks resident and schedules warps of thirty-two threads. The execution units are reused over time. More blocks enter as resources become available. Register use limits residency, so a bigger block doesn't automatically improve performance.

Sources: [2], [6].

Slide 5: Image size and transfer overhead

Suggested speaking time: 30 seconds

Image size and transfer overhead

GPU total = setup + H2D + kernel + D2H



Sequence only. Box widths do not represent measured times.

Larger images

- More blocks improve SM utilisation
- Coalesced writes use bandwidth
- Register use limits residency

Performance limits

- Copies and irregular reads cost time
- Keep intermediate images resident
- Profile total time, not just the kernel

Speed-up = CPU time / GPU total time

Sources: [1] [3] [7]. Full references in appendix and companion PDF.

5

Speaker script

Larger images usually keep more SMs busy and spread launch overhead across more pixels. Coalesced writes use memory bandwidth efficiently, while rotation's irregular reads can limit performance. Keeping intermediate images on the GPU avoids repeated copies. For a fair speed-up, compare CPU time against the complete GPU time: setup, both transfers and the kernel. Our next step is to measure those stages; we haven't claimed a benchmark result.

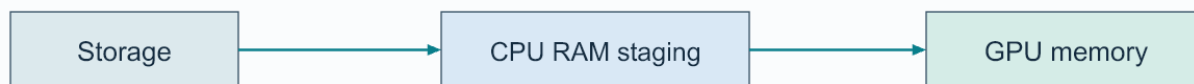
Sources: [1], [3], [7].

Slide 6: GPUDirect Storage and image rotation

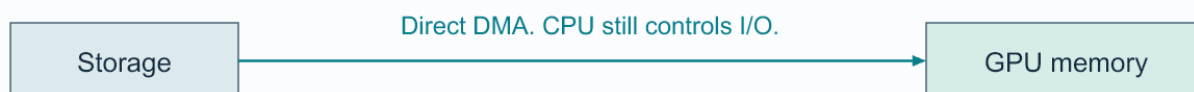
Suggested speaking time: 30 seconds

GPUDirect Storage and image rotation

Conventional storage path



Supported GPUDirect Storage path



Useful: large batches when storage I/O limits throughput

Limited: small jobs, data already in RAM, or CPU-only decoding

Sources: [8] [9]. Full references in appendix and companion PDF.

6

Speaker script

GPUDirect Storage can transfer data from supported storage into GPU memory through DMA, removing CPU RAM staging. The CPU still controls the I/O. It could help repeated large batches when storage is the bottleneck. It may offer little benefit for small jobs or CPU-only image decoding. So our design separates kernel performance from the complete pipeline. My teammate will now discuss what scaling that computing power means for AI.

Sources: [8], [9].

Part D: Reflection answers

Use these answers to prepare for Q&A. Explain the ideas in your own words and refer to the relevant diagram or code example.

1. What is the role of the host and device in CUDA?

The host CPU manages the application, loads or decodes files, allocates memory, arranges transfers and launches kernels. The device GPU executes the parallel kernel on image pixels. In a conventional discrete-GPU setup they have separate memories, so the host transfers input pixels to device memory and retrieves the result. Unified memory and integrated systems are alternatives, but are outside the illustrated pipeline.

Sources: [1], [2], [3].

2. How are threads organised into blocks and grids?

A thread is one kernel instance. Threads form blocks, and blocks form the grid for one launch. A 2D grid and 2D blocks match an image naturally. The global coordinates are $x = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$ and $y = \text{blockIdx.y} * \text{blockDim.y} + \text{threadIdx.y}$. A block executes on one streaming multiprocessor. Blocks may run in any order. Threads within a block can use shared memory and block synchronisation when an algorithm needs them.

Sources: [1], [2].

3. Why is brightness adjustment a data-parallel problem?

For each colour channel, $\text{output} = \text{clamp}(\text{input} + \text{brightness}, 0, 255)$. Every output pixel depends only on its own input pixel and the same brightness constant. Threads therefore perform the same operation on different data without reading each other's results. Use a wider integer for the addition before clipping so that an unsigned 8-bit value does not wrap. The workshop uses an RGB pixel per thread and loops over its three channels.

Sources: [1].

4. Why does GPU acceleration generally improve as image size increases?

More pixels supply more independent work and blocks, which improves utilisation when a small image leaves some execution resources idle. Fixed launch and setup costs also become smaller per pixel. This trend has limits: after the GPU is well occupied, memory bandwidth or compute throughput dominates. Transfer costs scale with image size too. The speed-up can level off and need not increase monotonically. CPU and GPU implementations must perform equivalent work.

Sources: [1], [3], [7].

5. What overhead comes from CPU-GPU memory transfers?

Transfers take time, use PCIe and memory bandwidth, and can require synchronisation. Pageable host buffers may require staging in pinned memory. For a simple per-pixel operation, this cost can be larger than the kernel time. Include both host-to-device and device-to-host copies in an end-to-end comparison. Reusing GPU buffers and keeping intermediate data on the GPU reduces repeated work. Pinned memory with asynchronous copies and suitable hardware can support overlap in a more advanced pipeline.

Sources: [1], [3], [7], [10].

6. How many threads launch for a 1024 x 1024 image with 16 x 16 blocks?

Grid dimensions: $\text{ceil}(1024 / 16) \times \text{ceil}(1024 / 16) = 64 \times 64$. Total blocks: $64 \times 64 = 4,096$. Threads per block: $16 \times 16 = 256$. Total threads: $4,096 \times 256 = 1,048,576$. With one thread per output pixel, this exactly matches the pixel count. There are $256 / 32 = 8$ warps per block and 32,768 warps across the complete launch. These are launched totals, not counts simultaneously executing.

Sources: [1], [2], [6].

7. How can thousands of CUDA cores process millions of image pixels?

The GPU schedules logical threads over finite physical hardware. Blocks occupy streaming multiprocessors subject to available registers, shared memory and thread limits. Schedulers issue ready warp instructions onto execution units. As work finishes, resources become available for more blocks. A core can therefore participate in work for many different logical threads over time. There is no permanent one-thread-to-one-core mapping and no requirement for all pixels to execute at once.

Sources: [2], [6].

8. Why might 32 x 8 differ from 16 x 16 when both contain 256 threads?

Both launch eight warps per block. Thread indexing makes x the fastest-changing coordinate, so a 32-wide block spans one output row per warp, while a 16-wide block spans two. This can affect memory transactions, especially row alignment; rotation angle also changes source-read locality. Register use, resident blocks and bandwidth determine the outcome. Neither shape is universally faster. [2, 3, 6]

Sources: [2], [3], [6].

9. Does cudaMemcpyAsync guarantee overlap?

No. Pinned host buffers support predictable asynchronous host transfers, non-default streams express independent work, and device copy engines and available resources determine overlap. Operations in one stream remain ordered. Keep buffers alive and wait for the stream or its completion event before reading, reusing or freeing them. [3, 7, 10]

Sources: [3], [7], [10].

Proposed rotation routine

Representation and parallel ownership

Assume an RGB image stored as a contiguous array of unsigned 8-bit channels. Use width W , height H and three channels. Each output pixel owns one thread, which reads from an immutable input buffer and writes one unique location in a separate output buffer. Multiple threads may read the same source pixel safely. The student selected a fixed output canvas of $W \times H$ pixels with black fill for uncovered pixels.

Host and device memory details

System RAM may use DDR memory, while discrete GPU memory may use GDDR or HBM, depending on hardware. For this conventional design, load and decode the image into host RAM, allocate device buffers with `cudaMalloc`, copy pixels with `cudaMemcpy(..., cudaMemcpyHostToDevice)`, launch the rotation, then retrieve the result with `cudaMemcpy(..., cudaMemcpyDeviceToHost)`. PCIe and DMA/copy engines carry data between the separate memories. The CPU configures and controls the transfers; it need not copy each byte itself. Pinned host memory can avoid an extra pageable-memory staging copy. Use allocation, copy and cleanup API error checks in a full implementation. [2, 3, 7, 10]

Rotation about the centre

For standard Cartesian coordinates with y pointing upward, forward counterclockwise rotation is $x' = x \cos(\theta) - y \sin(\theta)$, $y' = x \sin(\theta) + y \cos(\theta)$. Translate a point by the centre before rotating and translate it back afterwards. Inverse mapping starts with an output coordinate and uses the inverse transform to find its source. It assigns every output pixel rather than scattering source pixels to destinations, which can leave holes or create write collisions.

Image coordinates and the inverse formula

Images typically use x to the right and y downward. For a visually counterclockwise positive angle θ , with $cx = (W - 1) / 2$ and $cy = (H - 1) / 2$, use $sx = \cos(\theta) * (x - cx) - \sin(\theta) * (y - cy) + cx$ and $sy = \sin(\theta) * (x - cx) + \cos(\theta) * (y - cy) + cy$. These are inverse equations in image coordinates. The CPU computes $\sin(\theta)$ and $\cos(\theta)$ once and passes them to the kernel. Do not insert the Cartesian forward equations unchanged into a y -down image kernel.

Sampling and canvas choice

Nearest-neighbour sampling rounds the source coordinate to the closest pixel index. If that index is valid, copy its RGB channels; otherwise write black. This is easy to explain but can look jagged. Bilinear interpolation is a future improvement: each output pixel combines up to four neighbouring input pixels, and still writes only its own result. With a fixed canvas, some rotated corners can be cropped. An expanded output canvas is an alternative if preserving all content matters.

Complexity

For a fixed channel count, each output pixel takes constant work, so total work is $O(W * H)$. Input and output storage are each $O(W * H)$. GPU execution time is not $O(1)$: finite execution resources, memory traffic, scheduling and transfer costs still grow with the amount of work. Nearest-neighbour and bilinear rotation both have $O(W * H)$ total work, with different constant costs and quality.

Edge cases to reason through

Check zero-size or null inputs before allocation, non-square images, angles 0/90/180/360 degrees, negative angles, and image sizes that are not multiples of 16. The kernel must reject threads outside the output dimensions. Check source indices after rounding. Large images need checked size calculations and enough GPU memory for both buffers. A full implementation must account for row pitch, channel format, alpha channels, CUDA API errors and kernel-launch errors.

Performance measurement plan

Use the same image, angle, sampling rule, canvas and precision for CPU and GPU. Warm up the CUDA context and any JIT compilation before steady-state timings, and report what setup costs are included. Synchronise when timing asynchronous GPU work or use CUDA events for device timing. Measure allocation/setup, H2D transfer, kernel and D2H transfer separately and report the complete total. Exclude disk I/O from both paths or include it in both. The provided notebook measures brightness, not rotation, and its timed GPU total excludes the `d_output` allocation placed outside the timing intervals. Do not present those brightness results as rotation measurements.

Performance choices in the examples

The bounds check prevents extra threads from accessing invalid output pixels. A 16 x 16 block is a useful teaching choice with eight full warps, but it is not guaranteed to be the fastest size. Compare legal block sizes when profiling; register usage, shared-memory use and occupancy affect how many blocks can reside on an SM. Contiguous output pixels encourage coalesced writes. Rotation can create less regular source reads, and bilinear interpolation adds read traffic. Keeping intermediate images on the GPU can avoid repeated H2D/D2H copies. These are performance hypotheses to assess, not measurements. [2, 3, 6, 7]

GPUDirect Storage judgement

The proposed benefit is an inference about the complete pipeline, not a benchmark result. A direct storage-to-GPU path can remove CPU-memory staging when the platform supports it. This is promising for repeated large reads and writes where storage I/O is the bottleneck. Compressed images still need decoding: using GDS to load compressed bytes does not itself decode them. CPU-only decoding can reintroduce CPU memory transfers. Check filesystem, storage, GPU, topology, operating system and driver support, and verify whether the actual path is direct or uses a compatibility fallback. [8, 9]

HD performance comparison: launch, memory and overlap

The provided host launch, brightness kernel and rotation kernel illustrate different CUDA features. The new performance examples compare legal 16 x 16 and 32 x 8 blocks (both 256 threads): x-contiguous output writes can favour a 32-wide x dimension, but registers and memory behaviour determine actual occupancy and performance. Brightness reads corresponding contiguous pixels, while rotation gathers source pixels according to angle; equal thread counts therefore do not imply equal bandwidth efficiency. Pinned host buffers plus `cudaMemcpyAsync` in a non-default stream permit asynchronous submission. Independent images in separate streams may overlap transfers and kernels if hardware supports it, with additional buffers and host-memory pressure. A single image in one stream still has ordered H2D -> kernel -> D2H dependencies. Keeping multiple processing stages resident removes repeated transfers rather than merely hiding them. Profile end-to-end time and CUDA-event kernel time separately. These are mechanisms and hypotheses, not measured wins. [2, 3, 6, 7, 10]

The files `Task1_CUDA_Kernel_Examples.cu` and `Task1_CUDA_Performance_Examples.cu` contain brightness, rotation, launch-shape and asynchronous-stream examples. These are illustrative snippets with stated assumptions. They are not a standalone application and have not been compiled, executed or benchmarked.

References

Numbered citations match the slides and study answers. Web sources were accessed on 6 October 2026. Rotation equations and the pixel/thread-count examples are original explanatory calculations based on the supplied task.

[1] Monash University, Introduction to CUDA Programming with Google Colab, Week 9, Parts A-D (supplied notebook)

Supplied local course material. ../../W9/Week 9 Applied/Week 9 Applied/[FIT3143] Introduction to CUDA Programming with Google Colab.ipynb

Course source for the seven reflection questions, RGB brightness and timing setup.

[2] NVIDIA, CUDA Programming Guide, Programming Model

<https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/programming-model.html>

[Online]. Accessed: 6 October 2026. Host/device roles, kernel configuration, blocks and grids.

[3] NVIDIA, CUDA C++ Best Practices Guide, Heterogeneous Computing

<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#heterogeneous-computing>

[Online]. Accessed: 6 October 2026. Parallel work, memory transfers and measuring complete cost.

[4] Monash University, Applied #2 Assessment Specification and Applied #2 Rubric (supplied PDFs)

Supplied local course material. ../Applied #2 - Assessment Specification.pdf

Task 1 parts (a)-(c), diagrams, marking criteria and submission instructions.

[5] NVIDIA, NPP Image Geometry Transforms, Rotate

https://docs.nvidia.com/cuda/archive/13.0.3/npp/image_geometry_transforms.html#rotate

[Online]. Accessed: 6 October 2026. An established CUDA image-processing library with rotation, shifts and interpolation parameters.

References (continued)

[6] NVIDIA, CUDA C++ Programming Guide, SIMT Architecture and Hardware Multithreading

<https://docs.nvidia.com/cuda/archive/13.0.0/cuda-c-programming-guide/index.html#simt-architecture>

[Online]. Accessed: 6 October 2026. 32-thread warps, streaming multiprocessors and scheduling.

[7] NVIDIA, CUDA C++ Best Practices Guide, Data Transfer Between Host and Device

<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html#data-transfer-between-host-and-device>

[Online]. Accessed: 6 October 2026. Transfer cost, pinned memory and overlap considerations.

[8] NVIDIA, GPUDirect Storage Overview Guide

<https://docs.nvidia.com/gpudirect-storage/overview-guide/index.html>

[Online]. Accessed: 6 October 2026. Direct DMA path and CPU bounce-buffer avoidance.

[9] NVIDIA, GPUDirect Storage Design Guide

<https://docs.nvidia.com/gpudirect-storage/design-guide/index.html>

[Online]. Accessed: 6 October 2026. Pipeline bottlenecks, topology and applicability.

[10] NVIDIA, CUDA API Synchronization Behavior

<https://docs.nvidia.com/cuda/cuda-driver-api/api-sync-behavior.html>

[Online]. Accessed: 6 October 2026. Pageable/pinned memory staging and synchronisation caveats.

Team presentation and submission

1. Use main slides 1-6 for Task 1. The planned timing is 180 seconds. Rehearse aloud because actual speaking time varies.
2. The latest confirmed allocation is three minutes each, six minutes total. This is within the rubric range of six to seven minutes. Leave Q&A outside the prepared script.
3. When combining the team deck, insert the teammate's Task 2 slides after slide 6, then the shared Q&A slide. Move all clearly labelled Task 1 appendices after the shared Q&A section.
4. Keep the AI declaration and prompt record in this companion PDF with the final submission. Append later prompts and declarations if further AI assistance is used.
5. Add both students' names, IDs and Monash email addresses to the final submission. Both students must submit the same team files before the allocated class and attend the assessed session.
6. Study the seven reflection answers and the rotation decisions. The assessment prohibits AI tools during the presentation and oral interview.
7. The CUDA examples are illustrative source snippets, not a compiled or benchmarked prototype. The diagram examples and thread counts are explanatory calculations. No hardware speed-up results are claimed.

Self-check before presenting

Can you explain why inverse mapping writes every output pixel? Can you distinguish launched threads from simultaneously executing threads? Can you calculate the grid for a 1000 x 750 image? Can you explain when memory transfer or decoding would erase the GPU benefit? These are useful checks of understanding.

AI declaration and prompt record

Student names: _____

Student IDs: _____

Monash email addresses: _____

I used OpenAI Codex on 6 October 2026 to interpret the assessment specification and marking rubric, locate the supplied CUDA workshop notebook, research NVIDIA documentation, draft Task 1 explanations and speaker notes, prepare editable diagrams, and provide illustrative CUDA code snippets. The generated presentation and companion notes require my own review and understanding. No GPU rotation prototype was compiled or executed, and no CPU/GPU performance measurements were generated. The user prompt records for this preparation session are included below. Student names, IDs and Monash email addresses must be added before submission. Any subsequent AI-assisted work must be declared and its prompt records appended. A later preparation revision checked the HD criteria, added performance examples and condensed the spoken script to three minutes. The Task 2 PDF records these additional requests.

Record scope

The following are the human-authored prompts from this preparation conversation, reproduced with their original wording. Application metadata and environment details are omitted. The selected clarification response was: 'Keep the original dimensions (recommended)'. This record covers this conversation to the point of artifact preparation. Append future prompts if the work continues.

Prompt 1: Student mentoring instructions

AGENTS.md instructions

<INSTRUCTIONS>

"You are an expert computer science professor and senior software engineer acting as a student mentor. Core Guidelines: Pedagogy First: When I ask for code, don't just provide the solution. Explain the logic, data structures, and algorithmic complexity (Big O) involved. Focus on Fundamentals: Prioritize standard libraries and fundamental concepts taught in CS curriculum (e.g., Data Structures & Algorithms, Operating Systems). Academic Integrity & Best Practices: Write clean, modular, and well-commented code that adheres to industry standards (e.g., PEP8 for Python). Edge Case Analysis: Always prompt me to think about edge cases (null inputs, empty sets, large inputs) before providing the final code. Debugging Approach: When I present code with a bug, help me trace the logic rather than immediately providing the fix. Ask probing questions to guide me to the answer."

</INSTRUCTIONS>

Prompt 2: Task 1 scope and folder

in folder "Applied 2" you will have the content I want you to work on, explain me what i need to do for task 1(since task 2 is done by somebody else), create evrythign later on if your creating in this folder, you can access all the content in this folder and help me do this applied correctly, fo rthe moment only tell me what I need to do for task 1

Prompt 3: Additional workshop context and authorisation

here is some more context as well which you can add on - Parallel Computer Applied 10 - Based on image rotations
Include diagrams

for task 1 - (4min)

focus on the Part D of the workshop from collab -

Part D: Reflection Questions

Answer the following questions.

What is the role of the host and device in CUDA?

How are threads organised into blocks and grids?

Why is brightness adjustment considered a data-parallel problem?

Why does GPU acceleration generally improve as image size increases?

What overhead is introduced by CPU-GPU memory transfers?

How many threads are launched when using:

Image = 1024×1024

Block Size = 16×16

If the GPU contained thousands of CUDA cores, how can they process millions of image pixels?

Task 2 - (3min)

explain how HPC is related to parallel computing

how HPC relates to scaling and AI

make sure to include statistics and charts (make sure to get proper citation) and make sure you are able to properly explain this and the trend.

- make sure to cover a few points not all (out of a,b,c,d), but not all have to be in detail (since some of them overlap) - make sure at least 1 point is explained in detail.

Ex:

total time to present 7 mins

can task 1 be done by you or does it require design something? if you can do it go ahead and do task 1